
Docker Terminology

1. Docker Image

- A **read-only template** used to create containers.
- Contains everything needed to run an application:
 - Code
 - Runtime (ex: .NET, Node.js, Python)
 - Libraries
 - Environment settings
- Example: `mcr.microsoft.com/dotnet/aspnet:8.0`

Analogy: An *image* is like a recipe.

2. Docker Container

- A **running instance of an image**.
- Lightweight, isolated environment for your application.
- Multiple containers can be created from one image.

Analogy: A *container* is the actual dish cooked from the recipe.

3. Dockerfile

- A plain text file with step-by-step instructions to build a Docker image.
- Contains commands like:
 - `FROM`
 - `COPY`
 - `RUN`
 - `ENTRYPOINT`

Analogy: A *Dockerfile* is the recipe instructions you write.

4. Docker Engine

The core software that runs Docker. It includes:

- Docker Daemon
- REST API
- Docker CLI

It builds images, runs containers, and manages resources.

5. Docker Daemon (**dockerd**)

- The background service running on your machine.
- Listens for API requests.
- Builds and runs containers.

You don't interact with it directly — the CLI talks to it.

6. Docker CLI (**docker**)

- The command-line tool developers use.
- Examples:
 - `docker run`
 - `docker build`
 - `docker ps`

It sends commands to the Docker Daemon.

7. Docker Hub

- A cloud-based registry for sharing Docker images.
 - You can:
 - Pull official images (nginx, dotnet, python)
 - Push your own images
-

8. Registry

- A storage/service for Docker images.
- Could be:
 - Docker Hub
 - Azure Container Registry (ACR)
 - AWS Elastic Container Registry (ECR)
 - GitHub Container Registry

Analogy: A *registry* is like a storage shelf where recipes (images) are kept.

9. Repository

- A collection of related images, usually different versions/tags.
- Example:

- Repository: `nginx`
- Tags: `1.23`, `1.24`, `latest`

Analogy: A *repository* is like a folder of recipe variations.

10. Tag

- A label that represents a specific version of an image.
- Example:

- `dotnet:8.0`
 - `nginx:latest`
 - `myapi:v1.2`
-

11. Volume

- Persistent storage for containers.
- Containers are temporary—volumes allow you to keep data even if containers are deleted.
- Used for:
 - Databases
 - Logs
 - Uploaded files

Analogy: A *volume* is like a USB drive connected to your container.

12. Network

- Virtual network layer that allows containers to communicate.
 - Docker supports different network types:
 - bridge (default)
 - host
 - overlay (for Swarm/Kubernetes)
-

13. Build

- The process of reading a Dockerfile and producing an image.
 - Command: `docker build -t myapp .`
-

14. Run

- The action of starting a container from an image.
- Command: `docker run -p 8080:80 myapp`

15. Docker Compose

- A tool for running multi-container applications.
- Uses a `docker-compose.yml` file.
- Example use cases:
 - API + Database
 - Microservices

Command:

```
docker compose up
```

Quick Summary Table

Term	Meaning
Image	App template (read-only)
Container	Running instance of an image
Dockerfile	Instructions to build an image
Docker Engine	Runs and manages everything
Docker Daemon	Background process
Docker CLI	Command-line tool
Registry	Stores images
Repository	Collection of image versions
Tag	Version label of an image
Volume	Persistent storage
Network	Communication layer for containers
Build	Create image from Dockerfile
Run	Start a container
Compose	Tool to manage multiple containers
